Secure DevSecOps CI/CD Pipeline Implementation

Abubakar / Repository: kryptbakar

April 16, 2026

Project Objective

A complete integration of automated security gates directly into the development lifecycle, demonstrating the power of "Shifting Security Left."

1. Introduction

This project demonstrates the implementation of **DevSecOps** by integrating security directly into the Continuous Integration and Continuous Deployment (CI/CD) pipeline.

Traditionally, security is applied at the final stage of development, which leads to late detection of vulnerabilities and high remediation costs. This project solves that problem by implementing **Shift Left Security**, where vulnerabilities are detected early during development.

2. Core Concept

DevSecOps is the fusion of **Development (Dev)**, **Security (Sec)**, and **Operations (Ops)**.

> *"Security should be automated, continuous, and enforced—not manual or optional."*

In this implementation:

- **Continuous Scanning:** Every code push is automatically scanned.
- **Enforced Policy:** Vulnerable code is **blocked** before it can ever reach deployment.

3. Workflow Overview

The code follows a rigorous path from the developer's machine to production:

Phase 1: Developer Stage (Local)

The developer writes code using Python/Flask. To test the pipeline, vulnerabilities are intentionally introduced:

- **Hardcoded Secrets:** Storing API keys directly in the script.
- **Vulnerable Dependencies:** Using outdated libraries like `urllib3 1.25.8`.

Phase 2: CI/CD Pipeline Trigger

Once pushed to GitHub, Actions take over:

```
git commit -m "update code"
git push
```

The `devsecops.yml` workflow file initiates an isolated Ubuntu runner to execute the security checks.

4. Pipeline Stages (Security Gates)

1. **Stage 1: Build & Functional Testing (Status: PASS)**
   Tool: `pytest`. Ensures the app works as intended. Functional tests pass even if the code is insecure.
2. **Stage 2: SAST - Static Application Security Testing (Status: FAIL)**
   Tool: **Bandit**. Analyzes source code without execution. Detects the hardcoded secret and SQL injection points. **Result:** Pipeline blocks deployment.
3. **Stage 3: SCA - Software Composition Analysis (Status: FAIL)**
   Tool: `pip-audit`. Scans `requirements.txt` against known CVE databases. Detects *CVE-2020-26137*. **Result:** Pipeline blocks deployment.

5. Remediation Strategy

To pass the security gates, the developer must resolve the issues highlighted in the GitHub Action logs:

Problem	Solution
Hardcoded Secret	Use Environment Variables (`.env`)
SQL Injection	Use Parameterized Queries
Vulnerable Library	Update `requirements.txt` to latest versions

6. Advanced Usage

This pipeline is designed for scalability using two methods:

Method 1: Drop-In Method

Copy `.github/workflows/devsecops.yml` into any repository for instant security enforcement.

Method 2: Reusable Workflow

Reference the central pipeline dynamically:

```
jobs:
  call-security-workflow:
    uses: kryptbakar/Secure-DevSecOps/.../devsecops.yml@main
```

7. Real-World Importance

- **Cost Reduction:** Fixing a bug during development is 10x cheaper than fixing it after a breach.
- **Compliance:** Automates evidence gathering for SOC2 and ISO 27001.
- **Supply Chain Security:** Protects against malicious upstream packages.

8. Conclusion

This project proves that automated pipelines can act as an **Automated Security Guard**. By shifting security left, we ensure that only audited, secure, and compliant code reaches the production environment, significantly reducing the organization's attack surface.

Key Takeaways

- Shift Left Security detects issues early.
- SAST and SCA are essential for modern CI/CD.
- Automation removes the "human error" factor in security audits.